Cidades ESG Inteligentes

Desafio DevOps - Pipeline CI/CD, Docker e Deploy em Staging e Produção

Integrantes

Caio Agnello

Giovanna Vasconcelos Egawa

Sarah Schulze Bittar Bonafé

FIAP - Análise e Desenvolvimento de Sistemas
Fase 6 - Navegando pelo mundo DevOps
Outubro de 2026

Repositório: https://github.com/SarahSBB-Dev/cidades-esg-inteligentes

1. Descrição do projeto

A aplicação é uma API REST em **Java 21 com Spring Boot** que cadastra e consulta **indicadores ESG** de cidades, divididos nos pilares **Ambiental**, **Social** e **Governança**. Exemplos: percentual de energia renovável na iluminação pública, moradores atendidos por saneamento e transparência nos gastos públicos.

O objetivo do trabalho é automatizar todo o ciclo de vida da aplicação com práticas de DevOps. Cada alteração enviada ao GitHub é compilada, testada, empacotada em uma imagem Docker, publicada no Docker Hub e implantada automaticamente nos ambientes de **staging** e **produção**, onde a aplicação é verificada e testada.

Método	Rota	Descrição
GET	`/api/status`	Aplicação no ar e ambiente em que está rodando (dev, stg ou prd)
GET	`/api/indicadores`	Lista os indicadores (filtro opcional por categoria)
GET	`/api/indicadores/{id}`	Busca um indicador
POST	`/api/indicadores`	Cadastra um indicador (com validação dos campos)
PUT	`/api/indicadores/{id}`	Atualiza um indicador
DELETE	`/api/indicadores/{id}`	Exclui um indicador

2. Descrição do pipeline

Ferramenta: GitHub Actions, com o workflow `.github/workflows/ci-cd.yml`. As credenciais do Docker Hub ficam nos secrets do repositório (`DOCKERHUB_USERNAME` e `DOCKERHUB_TOKEN`).

Fluxo de branches: o desenvolvimento acontece em branches `feature/*`, integradas na `develop` por pull request. Um push na `develop` publica em **staging**. A `develop` validada é mesclada na `main`, que publica em staging e, em seguida, em **produção**.

#	Etapa (job)	Gatilho	Lógica
1	Build e testes	push e pull request (develop/main)	Configura o JDK 21 (Temurin), executa os testes automatizados com `mvn test` (JUnit 5) e gera o `app.jar` com `mvn package`.
2	Build e push da imagem Docker	push (develop/main), após a etapa 1	Constrói a imagem pelo Dockerfile (Docker Buildx) e envia ao Docker Hub com as tags `<sha do commit>` e `latest`.
3	Deploy em staging	após a etapa 2	No environment `staging`: cria o `.env` do ambiente, baixa a imagem do Docker Hub, sobe API + MySQL com o `docker-compose.yml` (`PROFILE=stg`), verifica o ambiente com `scripts/verificar-ambiente.sh` e testa a API com `scripts/testar-api.sh`.
4	Deploy em produção	após a etapa 3, só na `main`	No environment `production`: implanta **a mesma imagem** com `PROFILE=prd` e banco próprio, verifica o ambiente e testa a API.

Lógica de promoção: as etapas são encadeadas com `needs`. Se um teste falhar, nenhuma imagem é gerada. Se o deploy em staging falhar, a produção não é atualizada. Staging e produção recebem exatamente a mesma imagem (identificada pelo SHA do commit); entre os ambientes mudam só as variáveis de ambiente do `.env` (`PROFILE`, nome do banco e senha). Os jobs de deploy usam os **environments** `staging` e `production` do GitHub, que registram cada implantação na página *Deployments* do repositório.

Arquivo do pipeline (ci-cd.yml)

```
name: CI/CD - Cidades ESG Inteligentes

# Fluxo de branches: feature/* -> develop (staging) -> main (produção)
on:
  push:
    branches:
      - develop
      - main
  pull_request:
    branches:
      - develop
      - main
  workflow_dispatch:

env:
  # Nome do repositório da imagem no Docker Hub
  IMAGE_NAME: 'cidades-esg-inteligentes'

jobs:
  # ETAPA 1 - Build automático + testes automatizados
  build-test:
    name: Build e testes
    runs-on: ubuntu-latest
    steps:
      - name: Git Checkout
        uses: actions/checkout@v5

      - name: Setup Java SDK
        uses: actions/setup-java@v5
        with:
          distribution: 'temurin'
          java-version: '21'
          cache: 'maven'

      - name: Testes automatizados
        run: mvn -B test

      - name: Build do projeto (gera o app.jar)
        run: mvn -B package -DskipTests

  # ETAPA 2 - Build da imagem Docker e envio para o Docker Hub
  docker:
    name: Build e push da imagem Docker
    needs: build-test
    if: github.event_name != 'pull_request'
    runs-on: ubuntu-latest
    steps:
      - name: Git Checkout
        uses: actions/checkout@v5

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Log in to registry
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}
```

```
      - name: Build and push container image to registry
        uses: docker/build-push-action@v6
        with:
          push: true
          file: ./Dockerfile
          tags: |
            ${{ secrets.DOCKERHUB_USERNAME }}/${{ env.IMAGE_NAME }}:${{ github.sha }}
            ${{ secrets.DOCKERHUB_USERNAME }}/${{ env.IMAGE_NAME }}:latest

  # ETAPA 3 - Deploy automático no ambiente de STAGING
  deploy-staging:
    name: Deploy em staging
    needs: docker
    runs-on: ubuntu-latest
    environment:
      name: 'staging'
    steps:
      - name: Git Checkout
        uses: actions/checkout@v5

      - name: Criar o arquivo .env do ambiente
        run: |
          echo "PROFILE=stg" > .env
          echo "DB_NAME=esg_staging" >> .env
          echo "DB_ROOT_PASSWORD=$(openssl rand -hex 12)" >> .env
          echo "APP_IMAGE=${{ secrets.DOCKERHUB_USERNAME }}/${{ env.IMAGE_NAME }}:${{ github.sha }}" >>
.env

      - name: Log in to registry
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}

      - name: Baixar a imagem publicada no Docker Hub
        run: docker compose pull

      - name: Subir o ambiente (API + MySQL)
        run: docker compose up -d --no-build

      - name: Verificar se o ambiente está no ar
        run: bash scripts/verificar-ambiente.sh http://localhost:8080 stg

      - name: Testar a API no ambiente
        run: bash scripts/testar-api.sh http://localhost:8080

      - name: Containers em execução
        run: docker compose ps

      - name: Logs da aplicação
        if: always()
        run: docker compose logs api --tail 40

      - name: Encerrar o ambiente
        if: always()
        run: docker compose down -v

  # ETAPA 4 - Deploy no ambiente de PRODUÇÃO (somente na branch main e se staging deu certo)
  deploy-producao:
    name: Deploy em produção
    needs: deploy-staging
```

```
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    environment:
      name: 'production'
    steps:
      - name: Git Checkout
        uses: actions/checkout@v5

      - name: Criar o arquivo .env do ambiente
        run: |
          echo "PROFILE=prd" > .env
          echo "DB_NAME=esg_producao" >> .env
          echo "DB_ROOT_PASSWORD=$(openssl rand -hex 12)" >> .env
          echo "APP_IMAGE=${{ secrets.DOCKERHUB_USERNAME }}/${{ env.IMAGE_NAME }}:${{ github.sha }}" >>
.env

      - name: Log in to registry
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}

      - name: Baixar a imagem publicada no Docker Hub
        run: docker compose pull

      - name: Subir o ambiente (API + MySQL)
        run: docker compose up -d --no-build

      - name: Verificar se o ambiente está no ar
        run: bash scripts/verificar-ambiente.sh http://localhost:8080 prd

      - name: Testar a API no ambiente
        run: bash scripts/testar-api.sh http://localhost:8080

      - name: Containers em execução
        run: docker compose ps

      - name: Logs da aplicação
        if: always()
        run: docker compose logs api --tail 40

      - name: Encerrar o ambiente
        if: always()
        run: docker compose down -v
```

3. Docker

Arquitetura

Componente	Detalhe
Imagem da aplicação	Dockerfile em duas etapas: `maven:3.9.8-eclipse-temurin-21` compila e gera o `app.jar`; a imagem final usa só `eclipse-temurin:21-jre-alpine`. Porta 8080. O profile do Spring vem da variável `PROFILE`.
Ambiente local (Docker Compose)	Serviço `api` (Spring Boot) e serviço `db` (MySQL 8.0) na rede `esg-network` (bridge). A API acessa o banco pelo nome `db` e só inicia depois que o healthcheck do MySQL (`mysqladmin ping`) passa.
Volume	`db_data`: guarda os dados do MySQL mesmo se o container for recriado.
Variáveis de ambiente	Arquivo `.env` (modelo em `.env.example`): `PROFILE`, `DB_NAME`, `DB_ROOT_PASSWORD`. A API recebe `DATABASE_URL`, `DATABASE_USER` e `DATABASE_PWD`.
Staging e produção	O mesmo `docker-compose.yml` é usado nos environments `staging` (`PROFILE=stg`, banco `esg_staging`) e `production` (`PROFILE=prd`, banco `esg_producao`). A variável `APP_IMAGE` aponta para a imagem publicada no Docker Hub: `sarahsbb/cidades-esg-inteligentes:<sha do commit>`.

Dockerfile (imagem criada)

```
# Etapa de build: usa uma imagem com o Maven e Java 21 para compilar o projeto
FROM maven:3.9.8-eclipse-temurin-21 AS build
# Cria um diretório para armazenar os arquivos do projeto
RUN mkdir /opt/app
# Copia os arquivos do projeto para dentro do container
COPY . /opt/app
# Define o diretório de trabalho dentro do container
WORKDIR /opt/app
# Executa o build do projeto, gerando o arquivo JAR
# (os testes automatizados já são executados no job de CI do pipeline)
RUN mvn clean package -DskipTests

# Etapa de runtime: usa uma imagem leve com apenas o JRE para rodar o app
FROM eclipse-temurin:21-jre-alpine
# Cria o diretório da aplicação no ambiente final
RUN mkdir /opt/app
# Copia o JAR gerado na etapa de build para a imagem final
COPY --from=build /opt/app/target/app.jar /opt/app/app.jar
# Define o diretório de trabalho
WORKDIR /opt/app
# Define a variável de ambiente do profile (dev, stg ou prd)
ENV PROFILE=prd
# Expõe a porta 8080 (porta padrão da aplicação Spring Boot)
EXPOSE 8080
# Comando de inicialização, interpolando a variável PROFILE
ENTRYPOINT ["sh", "-c", "java -Dspring.profiles.active=$PROFILE -jar app.jar"]
```

docker-compose.yml

```
services:
  db: # Serviço de banco de dados
    container_name: esg-mysql
    image: mysql:8.0 # Imagem oficial do MySQL
    ports:
      - "3306:3306" # Expõe a porta padrão do MySQL
    environment:
      MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASSWORD} # Senha do usuário root (vem do arquivo .env)
      MYSQL_DATABASE: ${DB_NAME}               # Cria o banco de dados da aplicação
    volumes:
      - db_data:/var/lib/mysql # Volume para persistir os dados do banco
    networks:
      - esg-network
    healthcheck: # Verifica se o banco está pronto antes de subir a API
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 10s
      timeout: 5s
      retries: 10

  api: # Serviço da aplicação
    container_name: esg-api
    build: . # Usa o Dockerfile no diretório atual (ambiente local)
    image: ${APP_IMAGE:-cidades-esg-inteligentes:local} # Nos deploys do pipeline, usa a imagem
publicada no Docker Hub
    ports:
      - "8080:8080" # Mapeia a porta da aplicação
    depends_on: # Aguarda o banco de dados estar saudável
      db:
        condition: service_healthy
    environment:
      - PROFILE=${PROFILE} # Define o perfil da aplicação (dev, stg ou prd)
      - DATABASE_URL=jdbc:mysql://db:3306/${DB_NAME}?createDatabaseIfNotExist=true # URL de conexão com
o banco
      - DATABASE_USER=root # Usuário do banco
      - DATABASE_PWD=${DB_ROOT_PASSWORD} # Senha do banco
    networks:
      - esg-network

networks: # Rede interna: a API acessa o banco pelo nome do serviço "db"
  esg-network:
    driver: bridge

volumes: # Volume nomeado para persistência dos dados do banco
  db_data:
```

Comandos usados

```
copy .env.example .env              # cria o arquivo de variáveis (Windows)
docker compose up --build           # constrói a imagem e sobe API + MySQL
docker compose ps                   # lista os containers do projeto
docker compose logs -f api          # acompanha os logs da aplicação
docker compose down                 # para e remove os containers
docker build -t sarahsbb/cidades-esg-inteligentes:v1 .     # gera a imagem manualmente
docker push sarahsbb/cidades-esg-inteligentes:v1           # envia ao Docker Hub (no pipeline é
automático)
```

Estratégias: build em duas etapas (imagem final menor e sem código-fonte), profile definido por variável de ambiente (a mesma imagem serve para dev, stg e prd), `.dockerignore` para não copiar arquivos desnecessários, healthcheck do banco com `depends_on: condition: service_healthy` e testes executados no CI (por isso o Dockerfile usa `-DskipTests`).

4. Prints do pipeline rodando

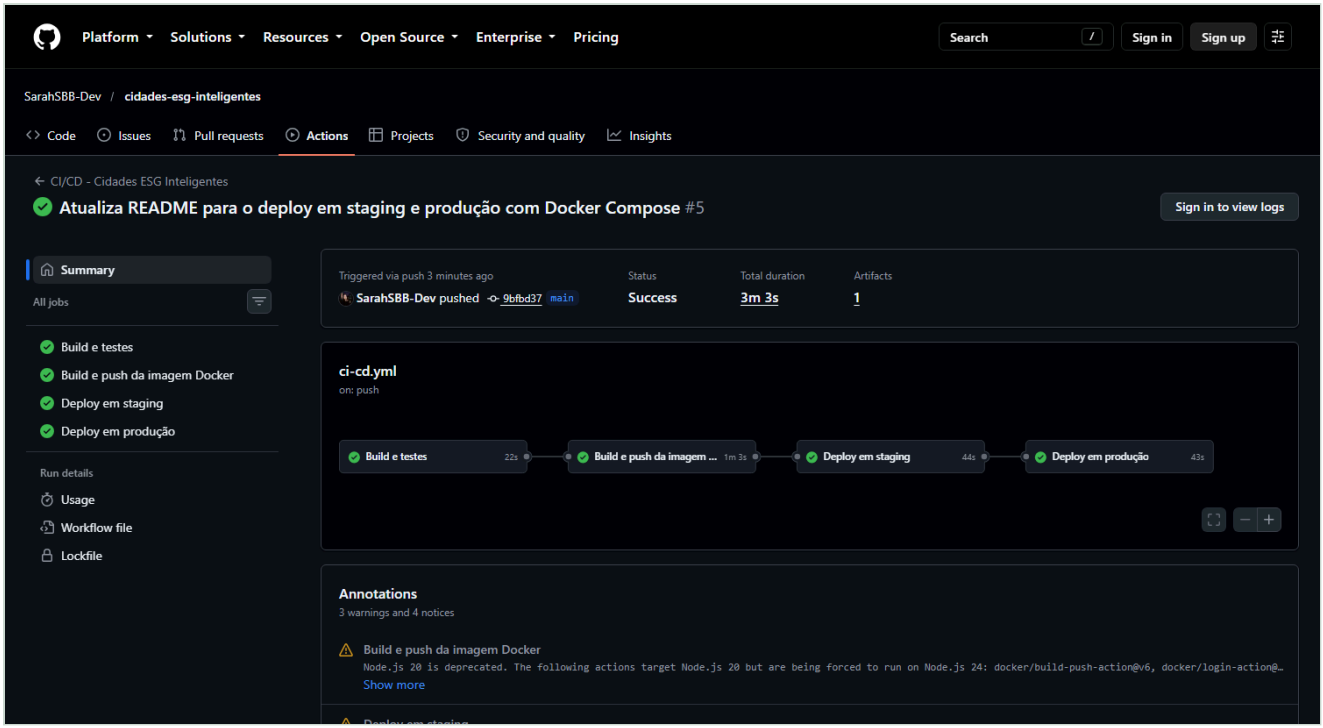


Figura 1 - Execução completa do pipeline no GitHub Actions: build e testes, imagem Docker, deploy em staging e deploy em produção

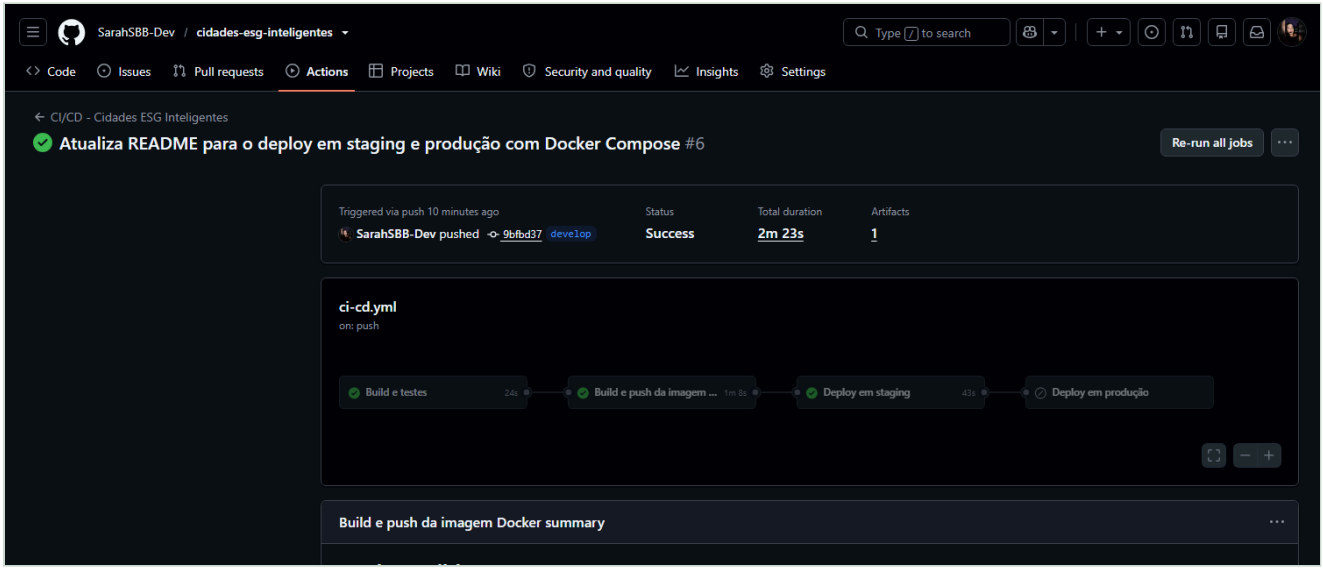


Figura 2 - Push na branch develop: o pipeline publica só em staging (o deploy em produção é pulado)

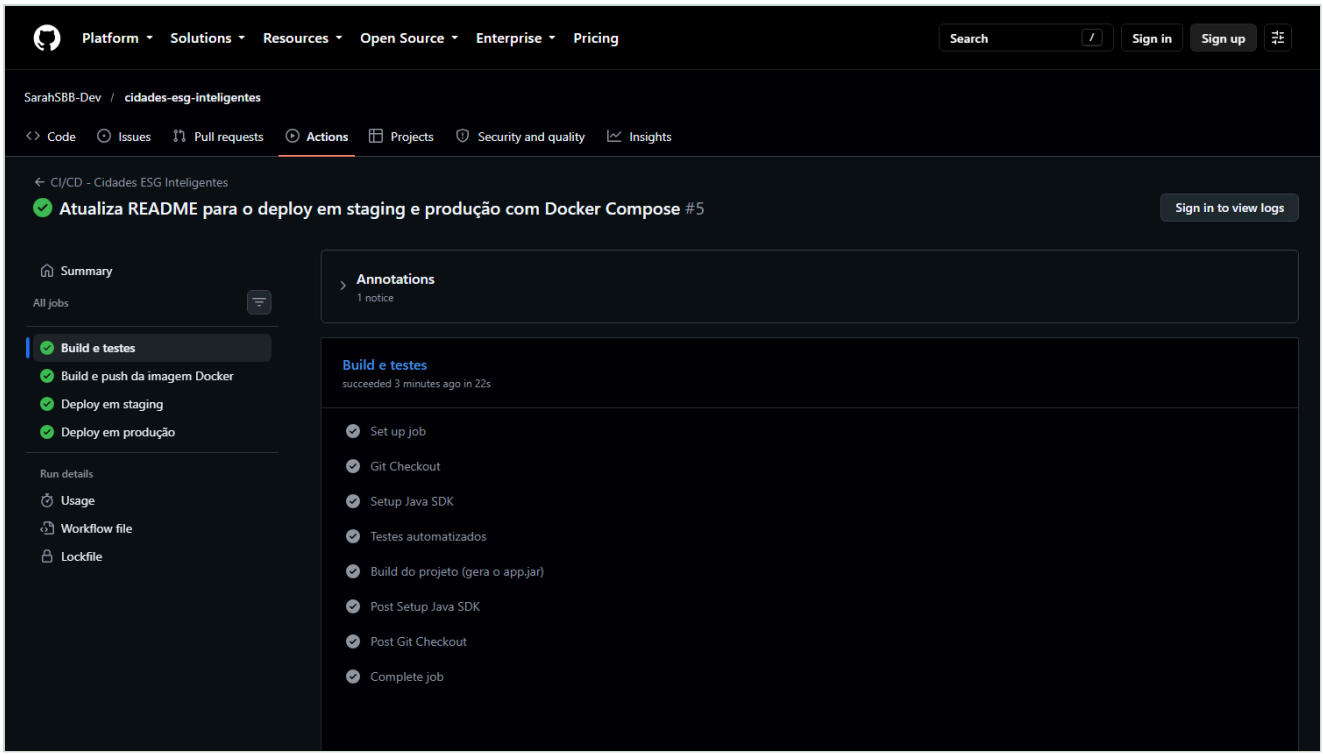


Figura 3 - Job "Build e testes": checkout, JDK 21, testes automatizados (mvn test) e build do app.jar

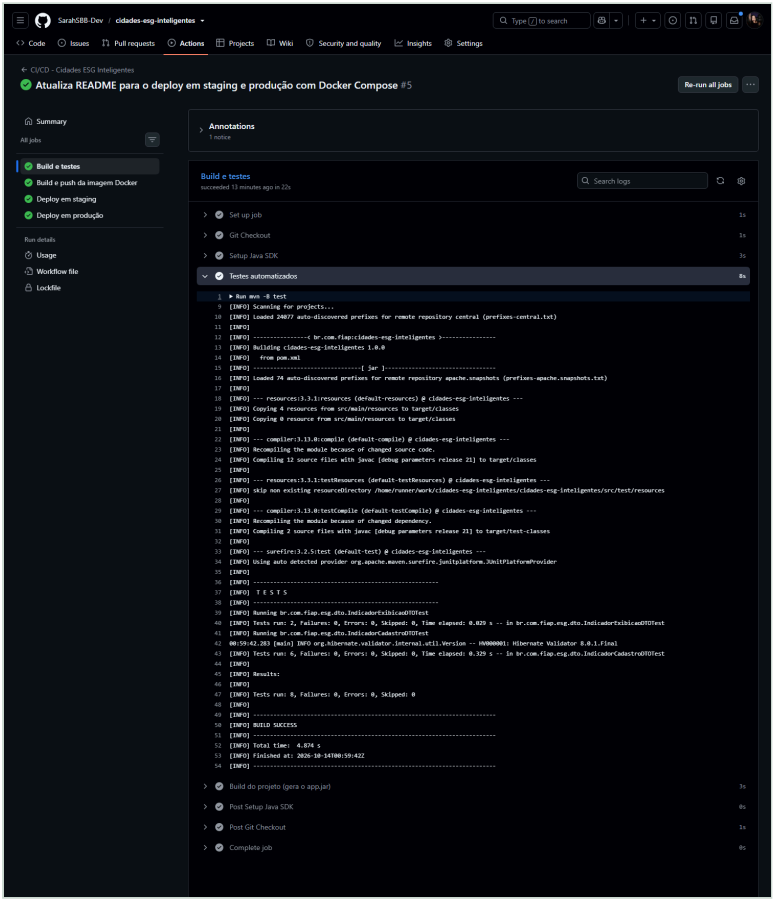


Figura 4 - Log dos testes automatizados (mvn test): Tests run: 8, Failures: 0, Errors: 0 - BUILD SUCCESS

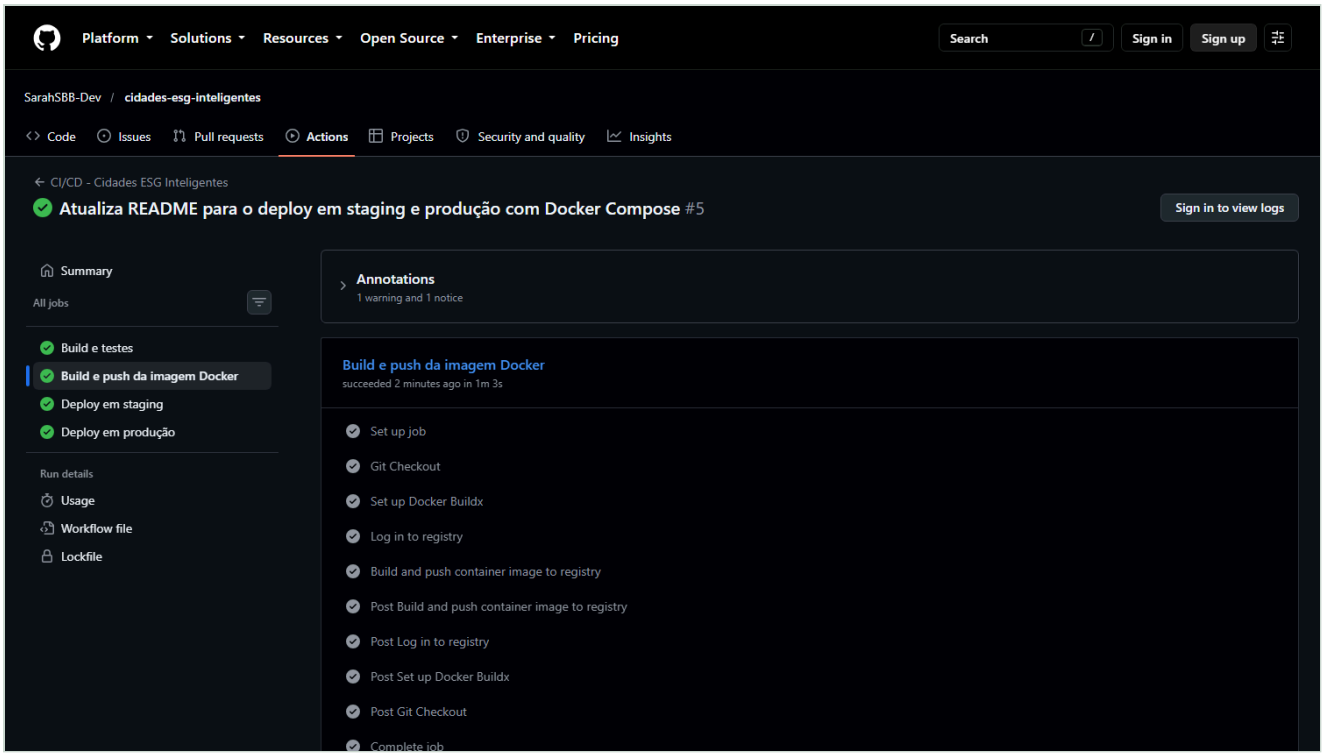


Figura 5 - Job "Build e push da imagem Docker": login e envio da imagem ao Docker Hub

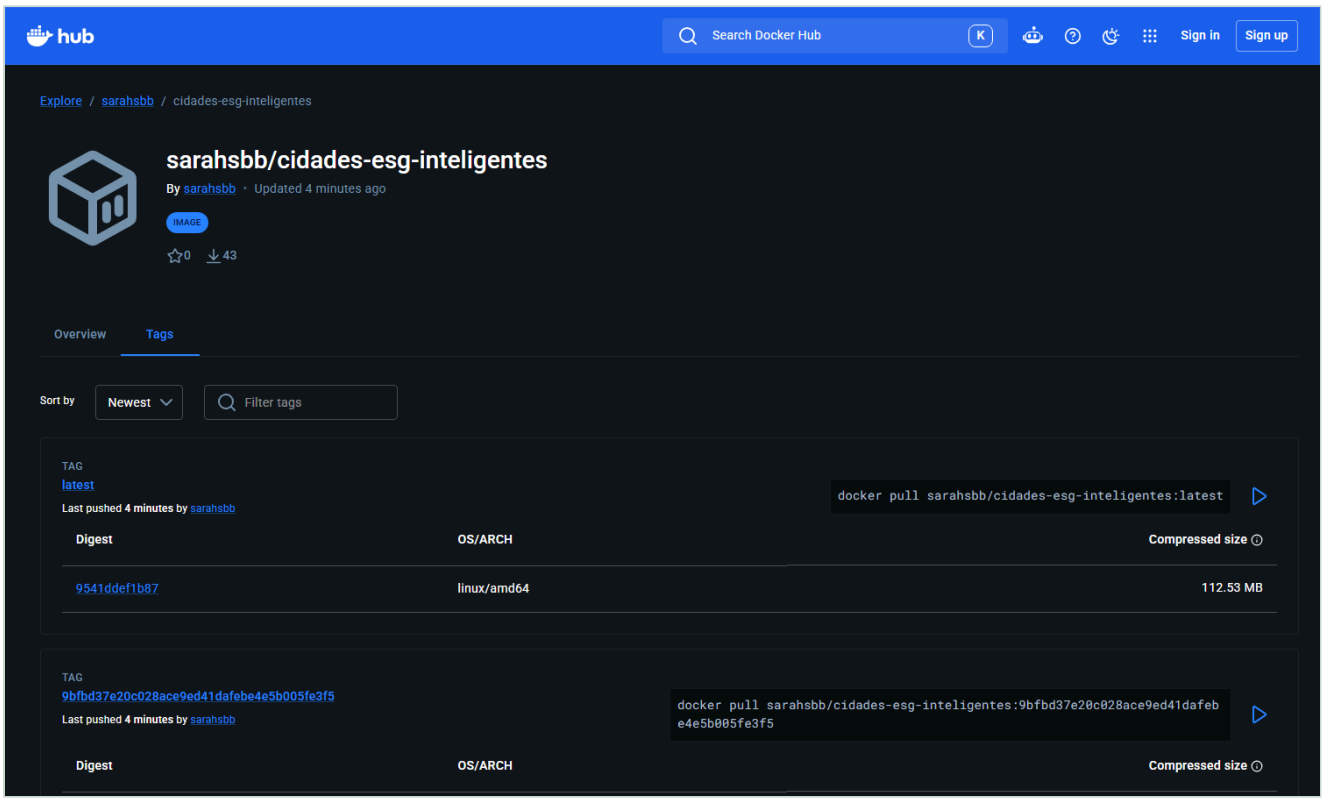


Figura 6 - Imagem publicada no Docker Hub com a tag do commit e a tag latest

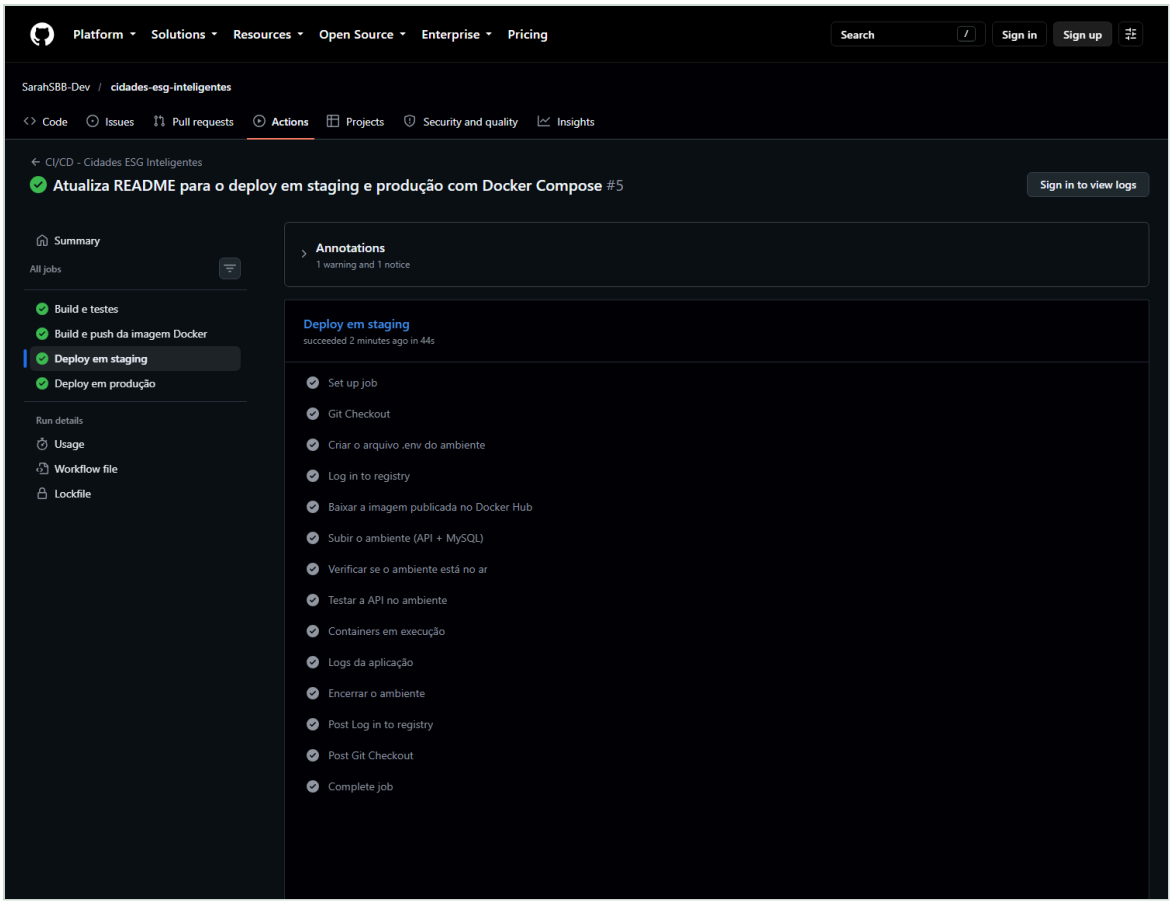


Figura 7 - Job "Deploy em staging": .env do ambiente, pull da imagem, docker compose up, verificação e teste da API

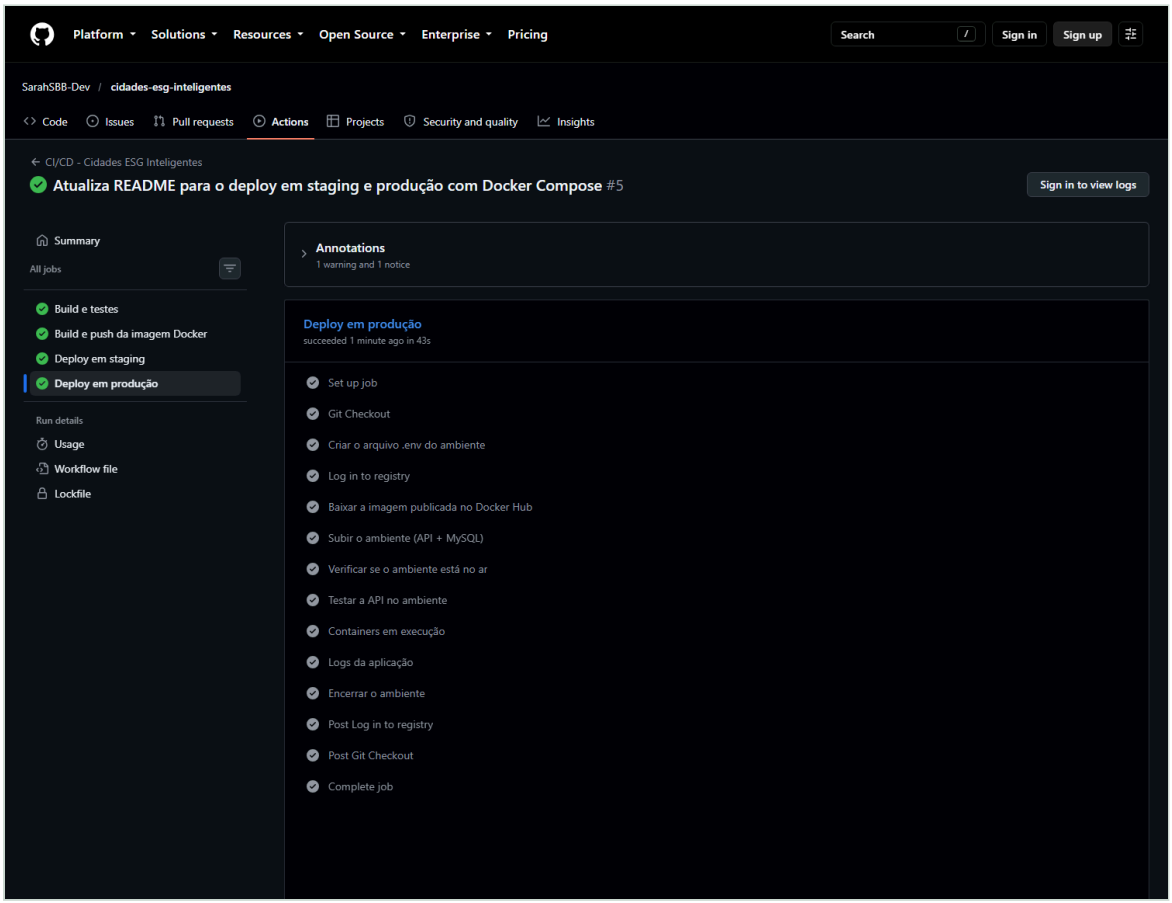


Figura 8 - Job "Deploy em produção": mesma imagem implantada no environment production

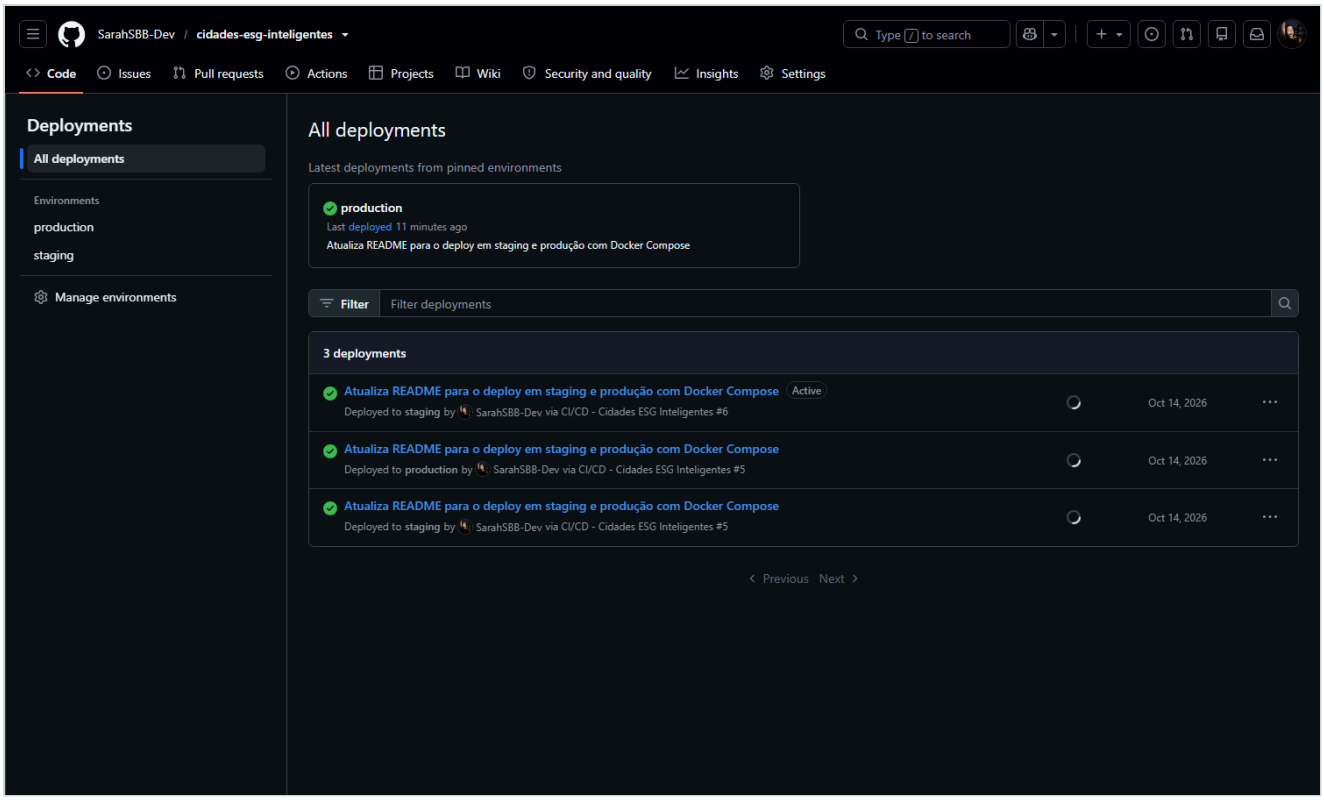


Figura 9 - Environments staging e production registrados no GitHub (histórico de deploys)

5. Prints dos ambientes staging e produção funcionando

Em cada ambiente, o pipeline mostra a resposta de `/api/status` com o profile correto, os cadastros feitos pela API (HTTP 201), a recusa de dados inválidos (HTTP 400), a listagem dos indicadores e os containers da API e do MySQL em execução.

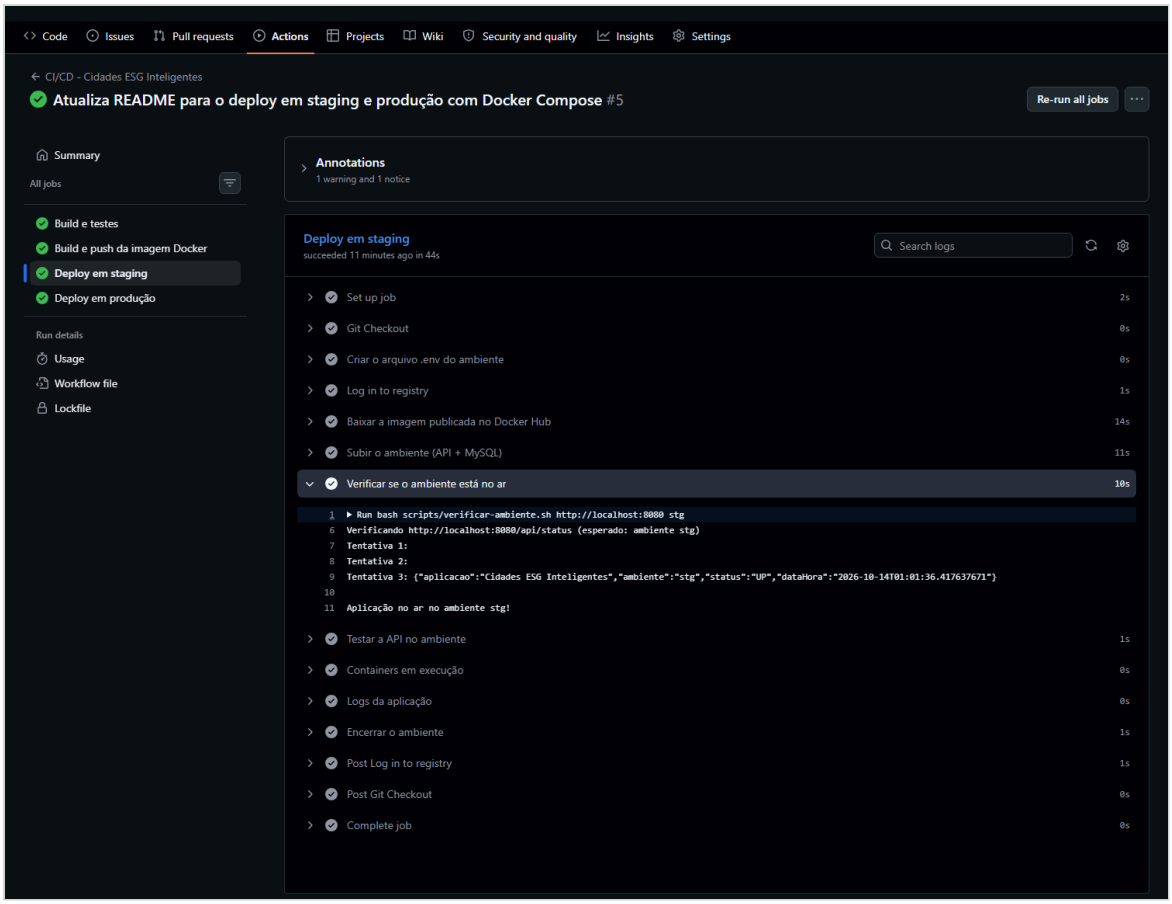


Figura 10 - Staging: aplicação no ar com "ambiente":"stg" em /api/status

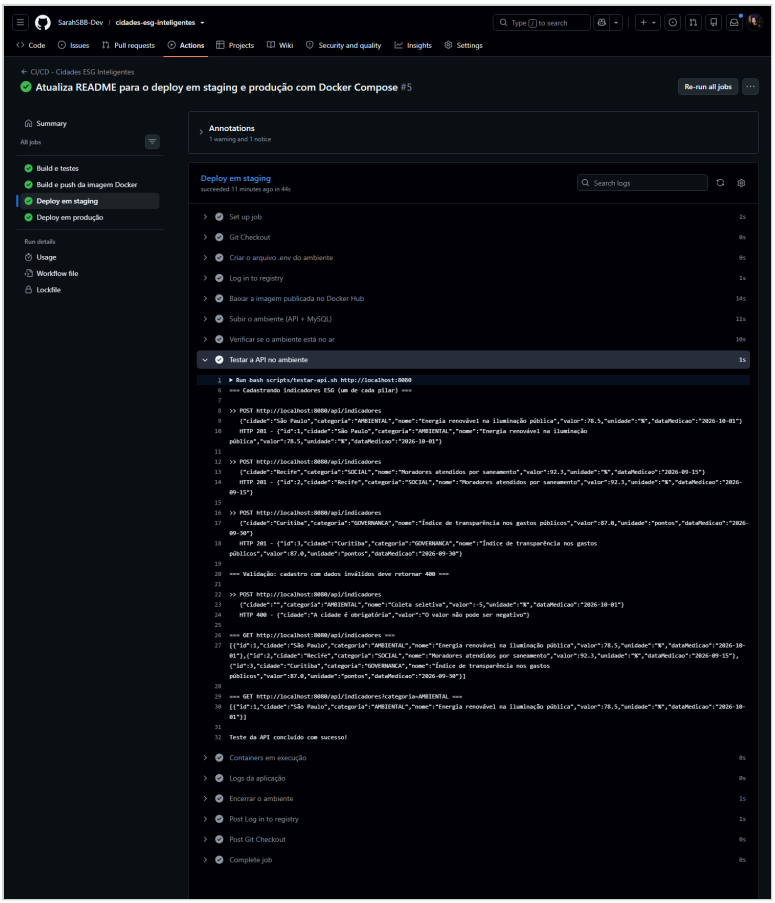


Figura 11 - Staging: cadastros (HTTP 201), validação (HTTP 400) e listagem dos indicadores ESG

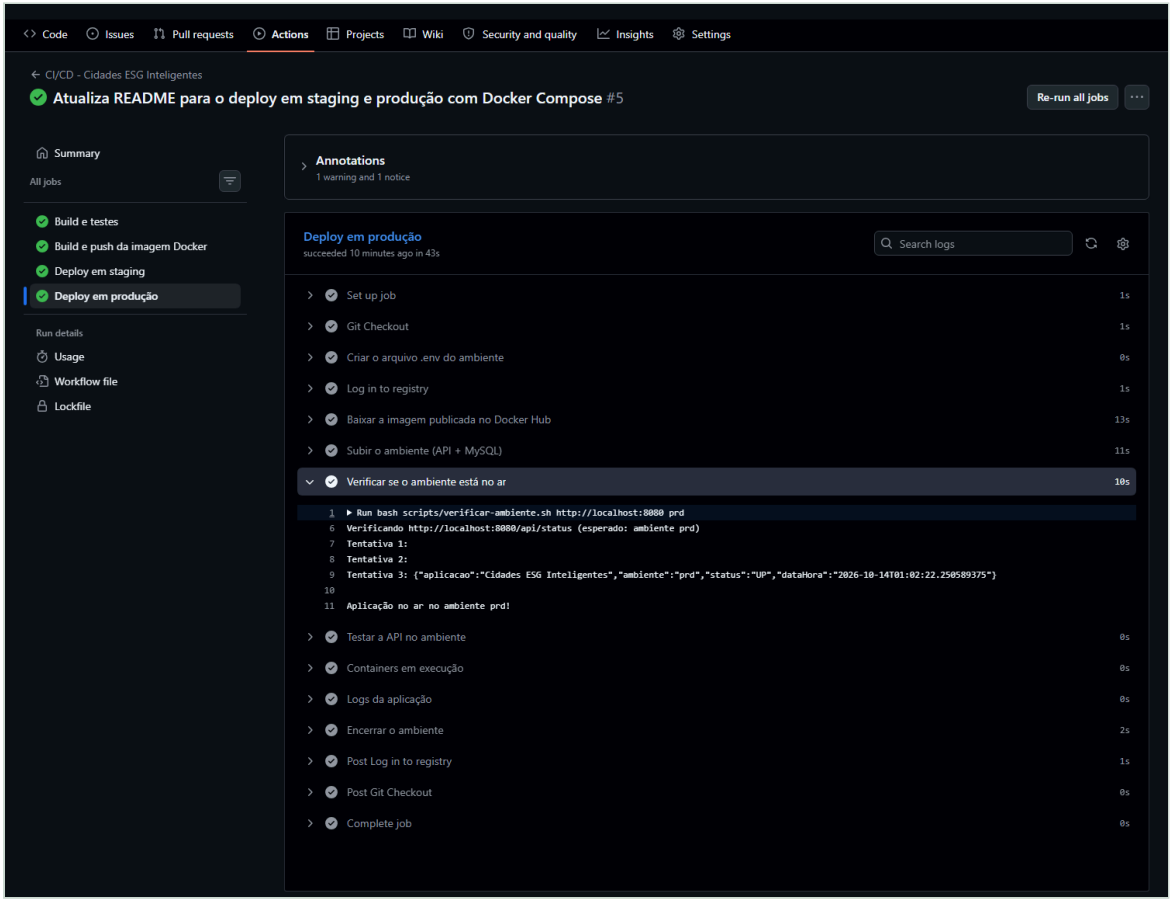


Figura 12 - Produção: aplicação no ar com "ambiente":"prd" em /api/status

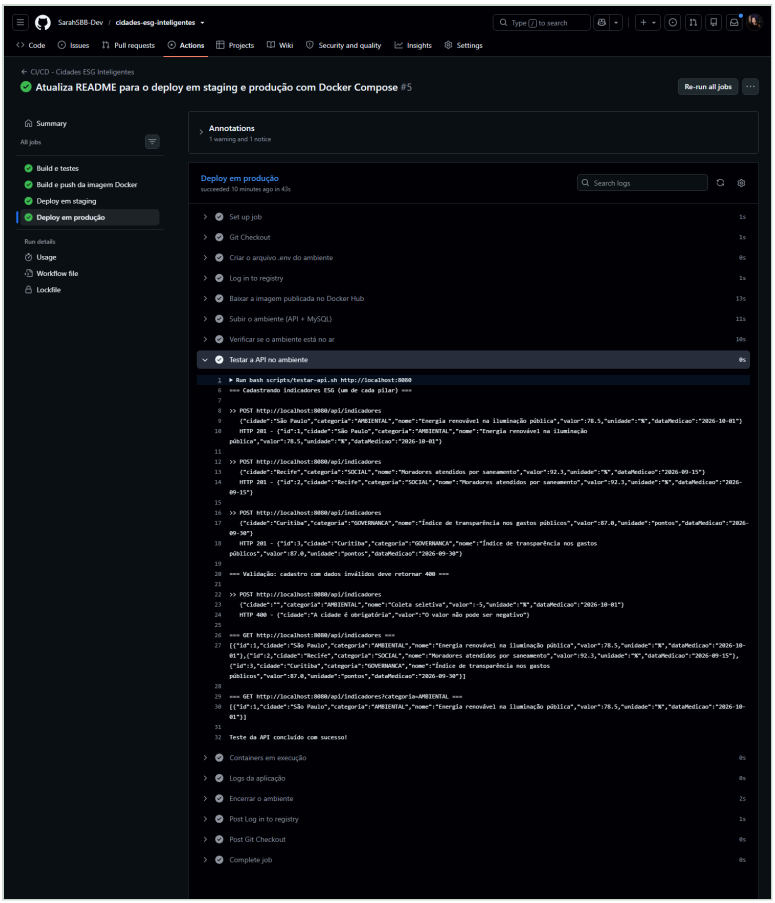


Figura 13 - Produção: cadastros (HTTP 201), validação (HTTP 400) e listagem dos indicadores ESG

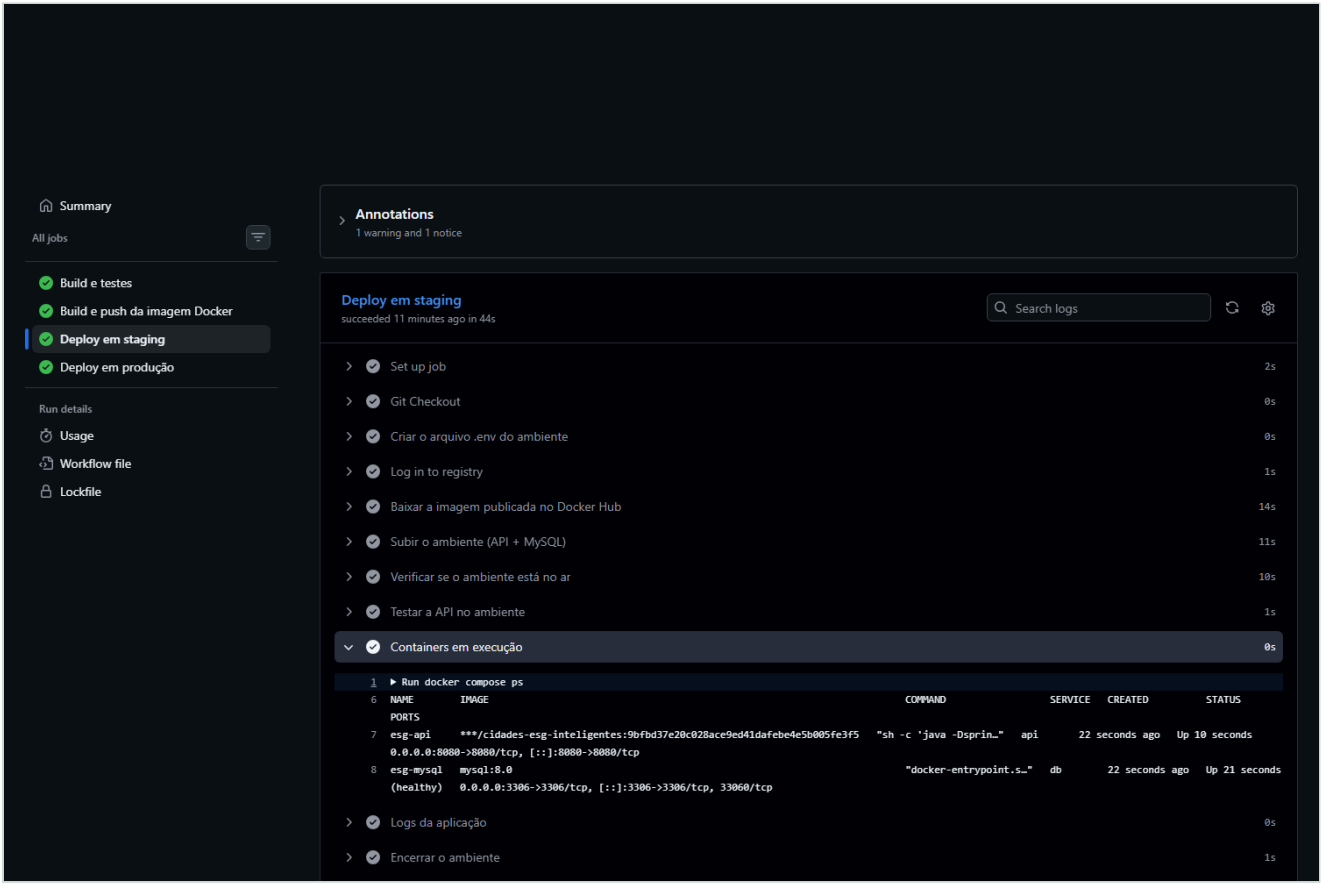


Figura 14 - Containers da API e do MySQL em execução no ambiente (docker compose ps)

6. Desafios encontrados e como foram resolvidos

Desafio	Solução adotada
O GitHub Actions não iniciou o workflow no primeiro push do repositório recém-criado.	Liberamos as actions em *Settings → Actions → General* (*Allow all actions and reusable workflows*) e fizemos um novo push, que disparou o pipeline.
A primeira execução do pipeline falhou no login do Docker Hub (*Username and password required*).	Cadastramos os secrets `DOCKERHUB_USERNAME` e `DOCKERHUB_TOKEN` no repositório. O token de acesso evita expor a senha no código.
O plano inicial era publicar staging e produção como Aplicativos Web no Azure, como no material da disciplina. Porém a conta Azure for Students bloqueou a criação dos recursos em todas as regiões testadas ("*Resource was disallowed by Azure: This policy maintains a set of best available regions...*").	Usamos os **environments** do GitHub Actions (`staging` e `production`) como ambientes de deploy. Em cada um, a imagem publicada no Docker Hub sobe com o mesmo `docker-compose.yml` (API + MySQL), é verificada e testada. Os scripts e o compose não dependem de provedor e podem ser reaproveitados em um servidor na nuvem.
O GitHub alertou que as actions em Node.js 20 estão descontinuadas e que `setup-java@v4` não recebe mais atualizações.	Atualizamos para `actions/checkout@v5` e `actions/setup-java@v5` .
O `docker-compose.yml` precisava construir a imagem localmente, mas usar a imagem já publicada nos deploys.	O serviço `api` tem `build: .` e `image: ${APP_IMAGE:-cidades-esg-inteligentes:local}` . Localmente a imagem é construída. No pipeline, `APP_IMAGE` aponta para a imagem do Docker Hub, baixada com `docker compose pull` e iniciada com `--no-build` .
Era preciso diferenciar staging e produção usando a mesma imagem.	Usamos profiles do Spring (`dev` , `stg` e `prd`) selecionados pela variável `PROFILE` , além de um banco diferente por ambiente. O endpoint `/api/status` mostra o ambiente ativo.
A senha do banco não pode ficar fixa no código dos ambientes.	O pipeline gera uma senha aleatória (`openssl rand`) ao criar o `.env` de cada ambiente. Localmente, a senha fica no `.env` , que não vai para o repositório (`.gitignore`).
A API tentava conectar antes de o MySQL estar pronto.	Adicionamos healthcheck no serviço `db` (`mysqladmin ping`) e `depends_on` com `condition: service_healthy` na API. Além disso, o script `verificar-ambiente.sh` espera a aplicação responder antes dos testes.

7. Tecnologias utilizadas

- **Linguagem e framework:** Java 21, Spring Boot 3.3 (Spring Web, Spring Data JPA, Bean Validation)
- **Banco de dados:** MySQL 8.0 (via Docker Compose em todos os ambientes)
- **Testes:** JUnit 5 (spring-boot-starter-test), executados com `mvn test`
- **Build:** Maven
- **Containerização e orquestração:** Docker e Docker Compose

- **Registro de imagens:** Docker Hub
- **CI/CD:** GitHub Actions, com os environments staging e production
- **Scripts:** Bash + curl (verificação do ambiente e teste da API)
- **Versionamento:** Git e GitHub (branches main, develop e feature/*)

8. Checklist de entrega

Item	OK
Projeto compactado em .ZIP com estrutura organizada	[x]
Dockerfile funcional	[x]
docker-compose.yml ou arquivos Kubernetes	[x]
Pipeline com etapas de build, teste e deploy	[x]
README.md com instruções e prints	[x]
Documentação técnica com evidências (PDF ou PPT)	[x]
Deploy realizado nos ambientes staging e produção	[x]